



Gestión de Pedidos en un Restaurante

Objetivos



Objetivo General

El trabajo práctico tiene por objetivo garantizar que cada alumno/a, a través de la investigación y experimentación, transite por el proceso de diseñar e implementar un sistema complejo.



Objetivos particulares

Principal

Que los alumnos/as adquieran, mediante el desarrollo del trabajo práctico, una mejor comprensión de los temas que se analizan en la materia desde una perspectiva teórica.

Trabajo en equipo

Que los alumnos/as comprendan la importancia del trabajo en equipo para llevar adelante un proyecto. Los alumnos/as deberán conformar equipos de 5 integrantes.

Técnicas y herramientas

Que los alumnos/as comiencen a utilizar, o incrementen sus conocimientos sobre, técnicas y herramientas de desarrollo de software actualmente utilizadas en la industria.

Forma de entrega y evaluación

El trabajo práctico debe estar versionado en un repositorio de código (github).

El trabajo práctico será evaluado tanto en forma grupal como en forma individual.

Evaluación grupal

Cada equipo debe realizar una presentación grupal del trabajo, en el cuál se evaluará el código y documentación de la solución.

Evaluación individual

Para la evaluación individual, serán tenidos en cuenta los aportes de cada integrante del equipo dentro del repositorio. En caso de que alguno de los integrantes del grupo no demuestre haberse comprometido con el trabajo grupal, dicho alumno/a deberá implementar funcionalidades extras al trabajo y luego se realizará un coloquio en el que deberá defender la solución propuesta.

Introducción

"Fogón del Barrio" es un restaurante que hasta ahora llevaba sus pedidos a mano, en papel. El dueño los contrató para digitalizar la **gestión de pedidos**, desde que el cliente los realiza hasta que la cocina los prepara y se genera la cuenta a pagar.

Etapla 1: Diseño e Implementación del Sistema de Gestión de Pedidos en un Restaurante

El restaurante atiende de tres formas distintas:

- **Pedidos en el salón**: el cliente se sienta en una mesa y pide directamente al mozo.
- **Pedidos para retirar**: el cliente encarga por teléfono y pasa a buscarlo en un horario acordado.
- **Pedidos con envío propio**: el restaurante cuenta con un cadete propio que reparte a domicilio dentro del barrio.

Cada una de estas modalidades tiene sus propias particularidades, por ejemplo: los **pedidos en salón** poseen una **mesa asociada**, los **pedidos para retirar** tienen **un horario de retiro**, y los **pedidos con envío** **una dirección y un costo de envío**. El dueño quiere en un futuro sea sencillo sumar nuevas modalidades sin tener que rehacer todo el sistema.

Menú, precios y combos

El **menú** se organiza en distintas categorías (entradas, platos principales, bebidas, postres, etc.). Cada producto individual tiene un precio de venta propio. Además de los productos individuales, el restaurante ofrece combos, que son agrupaciones de dos o más productos que se venden a un precio especial, menor a la suma de sus precios individuales. Ese "beneficio" del combo puede expresarse de distintas formas según el combo, por ejemplo, un descuento porcentual sobre la suma de sus productos, o directamente un precio fijo cerrado.

El dueño quiere poder crear combos nuevos con frecuencia sin que esto implique modificar el sistema cada vez.

Preparación en cocina y estaciones

El restaurante posee **varias estaciones** encargadas de preparar cada plato. Por ejemplo, la parrilla prepara las carnes, la cocina fría prepara ensaladas, la barra prepara bebidas, la cocina dulce prepara los postres. Un mismo **pedido** puede incluir productos que se preparen en distintas estaciones: un pedido con un plato de parrilla y un postre debe llegar a ambas estaciones para que cada una prepare lo que le corresponde, de forma independiente y **en el orden en que le fue llegando** a cada una. El dueño quiere que, si en el futuro agrega una **estación** nueva, esto no implique reescribir la lógica ya existente de las demás estaciones de trabajo.

Cada **ítem** de un **pedido**, va pasando por distintos estados a medida que se prepara (por ejemplo: **pendiente**, **en preparación**, **listo**). En esta primera versión alcanza con registrar en qué estado se encuentra cada ítem, respetando las siguientes reglas:

- Un **pedido** no **puede facturarse** hasta que todos sus **ítems** estén en estado **"listo"**.
- Un **ítem** que ya pasó a **"en preparación"** no puede modificarse ni cancelarse.



Corrección de pedidos antes de confirmar

Para los casos de toma de pedido en salón o para retirar, los ítems se van agregando al pedido de a uno. Mientras esto ocurre, se considera que el pedido se encuentra **"en construcción"**. Es común que, en este proceso, el mozo cargue algo por error (por ejemplo, un jugo de naranja en lugar de uno de manzana) y se dé cuenta antes de confirmar el pedido, al repasarlo con el cliente. En ese caso, el mozo necesita poder "borrar" lo escrito hasta alcanzar el ítem a corregir, sin tener que descartar todo el pedido y empezar de cero. Una vez que el **pedido** fue **confirmado** y enviado a cocina, ya no se permiten correcciones.



Medios de pago y descuentos

Al momento de generar la cuenta, el cliente puede pagar con distintos medios:

- Efectivo,
- Tarjetas: de débito, de crédito
- Billetera virtual.

El restaurante ofrece, además, algunos descuentos simples sobre el total del pedido:

- Un descuento por pagar en efectivo (por ejemplo, un porcentaje sobre el total).
- Un descuento especial en determinados días de la semana (por ejemplo, los martes).

En esta primera versión del sistema, un mismo pedido puede calificar para más de un descuento, pero solo se le aplica el que resulte más beneficioso para el cliente (nunca se combinan dos descuentos a la vez).

El dueño también quiere poder consultar en cualquier momento el historial de pedidos ya finalizados, con el detalle de qué se cobró en cada uno.

Requerimientos Funcionales

1. Registrar un pedido nuevo en alguna de las tres modalidades (salón, retiro, envío propio), con los datos particulares de cada una.
2. Agregar y quitar productos (individuales o combos) a un pedido mientras está "en construcción" (no confirmado).
3. Deshacer la última modificación realizada sobre un pedido en construcción.
4. Confirmar un pedido y distribuir sus ítems entre las estaciones de cocina que correspondan según los productos incluidos.
5. Permitir que cada estación procese sus ítems en el orden en que le llegaron, y marque el avance de cada uno (pendiente → en preparación → listo).
6. Impedir modificar o cancelar un ítem que ya está "en preparación".
7. Impedir facturar un pedido hasta que todos sus ítems estén "listos".
8. Calcular el precio total de un pedido, contemplando productos individuales y combos con su beneficio correspondiente.
9. Calcular el total a cobrar según el medio de pago elegido, aplicando el descuento por efectivo o por día especial cuando corresponda (solo el más beneficioso si se da más de uno).
10. Consultar el historial de pedidos ya finalizados, con el detalle de lo cobrado en cada uno.

¿Qué se pide?

1. Diagrama de clase de la solución propuesta.
2. Diagramas de secuencia de 2 o más requerimientos a definir por el equipo.
3. Desarrollar una aplicación que permita emular la situación planteada. Proveer el código y las pruebas unitarias para verificar los requerimientos funcionales.
4. Implementar una clase de integración que permita a la cátedra ejecutar, sobre el código propio del equipo, un conjunto de pruebas automatizadas estandarizadas. Los detalles de qué debe exponer esta clase, cómo debe nombrarse y qué restricciones tiene se encuentran en el **Anexo — Integración para evaluación automática**.

 **Sobre el Anexo:** además de este enunciado, la cátedra corregirá cada entrega ejecutando una batería de pruebas propia sobre una clase de integración que cada equipo debe entregar. Esa clase actúa como punto de traducción entre las pruebas de la cátedra y el diseño interno del equipo: solo delega en las clases del dominio, sin contener lógica propia. El diseño interno (qué clases existen, cómo se relacionan, qué colecciones se usan) sigue siendo una decisión exclusiva de cada equipo; el Anexo únicamente define el contrato de métodos que la clase de integración debe cumplir para que la corrección automática pueda ejecutarse. Se recomienda leer el Anexo en conjunto con este enunciado antes de comenzar el diseño.

Requerimientos adicionales:

- Desarrollar las pruebas unitarias para cada método realizado.
- El porcentaje de código cubierto por pruebas unitarias debe superar el 80%.
- Agregar archivo README.md al proyecto describiendo el proyecto, agregar instrucciones para su compilación y funcionamiento.
- Generar documentación de código utilizando [JSDoc](#).



continuará... 😏